Greedy and Divide-and-Conquer Algorithms for Real-World Problems

Ilan Danial
Dept. of Computer & Information Science & Engineering
University of Florida
Gainesville, FL, USA
ilan.danial@ufl.edu

Ryan Rodriguez
Dept. of Computer & Information Science & Engineering
University of Florida
Gainesville, FL, USA
rrodriguez7@ufl.edu

***Abstract*—A cruise-fare tracker that reads an unofficial, rate-limited search API should call it as rarely as it can. We model quote requests with acceptable date windows as integer intervals and choose the fewest refresh days that hit every window, a minimum point cover that a greedy scan by closing day solves optimally in $O(n \log n)$ time; on 383 real Carnival sailings it needs three refreshes. Map-search sites cap the number of listings shown per window, so collecting every listing in a city means choosing windows that never overflow. We model this as a point set on an integer grid and give a divide-and-conquer algorithm that recursively splits overflowing windows into quadrants. We prove it correct and bound its recursion depth by $\lceil \log_2 W \rceil$ for a map of W grid units; every statement is machine-checked in Lean 4. For evenly spread listings it makes $\Theta(n/k)$ requests and $\Theta(n \log(n/k))$ work, which experiments confirm on synthetic data and on 261 real Airbnb listings.**

***Index Terms*—greedy algorithm, interval point cover, web scraping, rate limiting, cruise fares, divide and conquer, quadtree, formal verification**

I. Introduction

This report applies two algorithm-design techniques to problems that arose outside the classroom. For each we state the problem, abstract it, give an algorithm with a running-time analysis and a proof of correctness, explain the algorithm in the problem's own terms, and check the analysis experimentally.

Both problems come from scraping travel marketplaces under a limit the platform imposes. The divide-and-conquer half asks *where* to look when a map caps the results per window; the greedy half asks *when* to look when an API caps how often one may ask. Section II takes a cruise-fare tracker that reads Carnival's unofficial, rate-limited search endpoint. We propose letting users ask for one fresh quote per sailing within a window of days, and choose the fewest refresh batches that serve every window. This is the minimum point cover of intervals. A greedy scan by closing day is optimal, which we prove with a set of pairwise disjoint witness windows, and runs in $O(n \log n)$ time, which experiments confirm up to $n = 2^{22}$. On 383 real sailings it needs three refreshes over six months.

The second problem comes from web data collection. A map-search service shows at most k listings for any map window, so a scraper that needs every listing in a city must split the map into windows that never overflow, using as few requests as it can. Section III models the map as an integer grid and the service as a capped oracle, solves the problem with a divide-and-conquer recursion on quadrants, proves the recursion correct and bounds its depth by $\lceil \log_2 W \rceil$, and verifies the predicted $\Theta(n \log(n/k))$ cost experimentally. Every statement in the correctness section is also machine-checked in Lean 4 (Appendix B).

II. Greedy: Fewest Price Refreshes Under a Request Budget

A. Problem

Fare trackers learn of price changes only by querying the seller's site, as in early airfare-prediction work [1], and a site may refuse requests that arrive too fast [2]. Cruise Finder [3] is a self-hosted web application, written by the second author, that tracks Carnival Cruise Line sailings from Florida. It reads prices from Carnival's public but undocumented search endpoint, calling it once per configured departure month and following every result page. Because the endpoint is unofficial and rate-limited, the application calls it only on demand and logs every call in an append-only audit table. We call one pass over every configured month and every page a *refresh batch*; it observes the current price of every listed sailing. We assume every batch costs the same, so the objective counts batches, not HTTP requests (the number of pages can change over time).

We propose a feature that the application does not have today: *quote requests*. A quote request names a sailing and a window of days during which one fresh observation of its price is acceptable. The question is: given every request's window, what is the fewest refresh batches that give every window at least one observation, and on which days?

For the case study we generate one request per sailing by a stated policy, a modeling choice rather than a claim about how fares move. Each window opens at Carnival's final-payment deadline, 76 days before departure for cruises of up to 5 days and 91 days for 6 days or more [4], and closes when booking closes, c days before departure. Final payment is when every booking must be paid in full; unpaid bookings are cancelled and their cabins return to sale, so it is a natural moment to want a current quote. We found no Carnival page giving the booking cutoff, so $c = 2$ is an assumption. The model asks for one observation per window, not continuous tracking, so it

```
1: function REFRESH-DAYS(W)
2: sort W so that b_1 ≤ b_2 ≤ ··· ≤ b_n
3: T ← ∅; S ← ∅; last ← −∞
4: for i = 1 to n do
5:    if a_i > last then
6:       T ← T ∪ {b_i}; S ← S ∪ {W_i} {greedy choice}
7:       last ← b_i
8:    end if
9: end for
10: return T, S {refresh days, witnesses}
```

Fig. 1. Fewest days hitting every window. S is used only in the proof.

minimizes refresh batches, not missed discounts or the average staleness that crawler refresh policies optimize [5].

B. Abstraction

Days are integers. Sailing i departs on day d_i and lasts L_i days; its window is $W_i = [a_i, b_i]$ with

$$a_i = d_i - F(L_i), \quad b_i = d_i - c, \quad F(L) = \begin{cases} 76 & L \le 5, \\ 91 & L \ge 6. \end{cases} \tag{1}$$

We take L_i from the search data's duration field, which equals arrival minus departure in days for every sailing we fetched and matches the "L-Day" length the table in [4] uses. Windows with b_i before today are dropped, and a_i is raised to today if it is earlier. A batch on day t observes every window with $t \in W_i$.

The abstract problem: given a set $\mathcal{W} = \{W_1, \ldots, W_n\}$ of integer intervals with $a_i \le b_i$, find a set of days $T \subseteq \mathbb{Z}$ that *hits* every interval ($T \cap W_i \neq \emptyset$ for all i) with $|T|$ minimum. This is the minimum point cover of intervals, also called interval stabbing, equivalent to a minimum clique cover of the interval graph [6]. It is not interval scheduling, which picks a largest set of pairwise disjoint intervals; here we pick points that meet every interval.

C. Algorithm

REFRESH-DAYS (Fig. 1) scans the windows by closing day. Whenever a window holds no chosen day yet, it chooses that window's last day, the latest day that still serves it. The windows that cause a choice are the *witnesses*. On the windows $[1, 10]$, $[4, 12]$, $[8, 20]$, $[15, 25]$, $[22, 30]$ it chooses days 10 and 25 with witnesses $[1, 10]$ and $[15, 25]$. One day cannot suffice, since $[1, 10]$ and $[22, 30]$ share no day.

D. Correctness

The proof follows the structural-bound pattern for greedy algorithms [7]: the algorithm's choices exhibit m windows that force any solution to use m days. Throughout, windows are indexed in sorted order, and *last* is the most recently added day, so $last = \max T$ at every step.

Lemma 1 (every window is covered)*: When* REFRESH-DAYS *returns, every W_i contains a day of T.*

Proof: Consider the step that processes W_i. If $a_i > last$, line 6 adds $b_i \in W_i$ to T. Otherwise $last \neq -\infty$, so $last = b_j$ for some earlier $j < i$, and $b_j \le b_i$ by the sort. Then $a_i \le b_j \le b_i$, so $b_j \in T$ lies in W_i. Days are never removed from T. ∎

Lemma 2 (witnesses are disjoint)*: The witnesses are pairwise disjoint, and $|S| = |T|$.*

Proof: Each addition puts one window in S and one day in T, and the days added are strictly increasing ($b_i \ge last$ and $b_i \ge a_i > last$), so $|S| = |T|$. Let $i_1 < \cdots < i_m$ be the indices that triggered additions, and take $r < s$. When i_s is processed, $last = b_{i_{s-1}} \ge b_{i_r}$, because *last* only increases and the b's are sorted. The test on line 5 passed, so $a_{i_s} > last \ge b_{i_r}$. Every day of W_{i_s} is at least a_{i_s} and every day of W_{i_r} is at most b_{i_r}, so the two windows share no day. ∎

Theorem 1 (optimality)*: T hits every window, and every set T' that hits every window has $|T'| \ge |T|$.*

Proof: Feasibility is Lemma 1. Let T' hit every window. It contains a day in each of the $m = |S|$ witnesses, and since the witnesses are pairwise disjoint (Lemma 2), no day lies in two of them. These m days are therefore distinct, so $|T'| \ge m = |T|$. ∎

The same argument shows that S is a largest set of pairwise disjoint windows, since any k disjoint windows need k distinct days of T. That the two numbers are equal is a known property of interval graphs [8].

Lemmas 1 and 2 and Theorem 1 are machine-checked in Lean 4 [9] (Appendix E), under the following correspondence:

Lemma 1	`refresh_covers`
Lemma 2	`witnesses_pairwise_disjoint`
Theorem 1	`refresh_optimal`

The Lean `refresh` is the loop of Fig. 1 on a list already sorted by closing day, carrying *last* as an argument; the sort itself, the window construction (1) and Theorem 2 are not formalized. The Lean proof of Lemma 2 does not need the sort: each witness opens after the previous one closes in any input order.

E. Running Time

Theorem 2 (running time)*:* REFRESH-DAYS *runs in $O(n \log n)$ time with a comparison sort and uses $O(n)$ extra space. If the windows arrive sorted by closing day, it runs in $O(n)$ time.*

Proof: Line 2 is a comparison sort, $O(n \log n)$ time and $O(n)$ space for the sorted copy. The loop runs n times, and each iteration does one comparison and at most two constant-time insertions (appending to lists, as days arrive in increasing order). T and S hold at most n items each. Without line 2, only the $O(n)$ loop remains. ∎

Theorem 2 is an upper bound. The sort is the only superlinear step, so the measured time should grow like $n \log n$ unless the input is pre-sorted.

F. In the Language of the Problem

Among the sailings not yet observed, look at the one whose quote window closes soonest and refresh on its last open day. That batch also observes every other sailing whose window is open that day; cross them off and repeat. No schedule with fewer batches exists, because the sailings we refreshed *for* have windows that never overlap, so each one needed a batch of its own. For the 383 Carnival sailings from Miami, Port Canaveral, Tampa and Jacksonville departing November 2026 through April 2027 (fetched October 26, 2026), three batches suffice: on October 30, 2026, January 14, 2027 and March 30, 2027, over a 184-day horizon (Fig. 3). Each sailing is observed once within its window, not nightly, so this is three calls of the full search instead of one per day.

G. Experiments

Setup. REFRESH-DAYS was implemented in Python 3.12.2 (Appendix D) using the built-in sort, Timsort with the powersort merge policy since Python 3.11 [10], and run on an AMD Ryzen 7 9800X3D, following the protocol of Section III-G. For $n = 2^{10}, \ldots, 2^{22}$ windows, departure days were drawn uniformly over a horizon of $2n$ days, so that closing days are mostly distinct; a fixed two-year horizon would make most windows tie. Cruise lengths were drawn from the 383 real sailings and windows built by (1) with $c = 2$. Each size was run five times on fresh inputs; we report the fastest run with garbage collection disabled, as `timeit` does [11]. Every run was checked to hit every window (Lemma 1) with one disjoint witness per day (Lemma 2). We also timed the scan alone on windows already sorted by closing day. Unit tests additionally compare the output with brute-force search on 300 random instances of up to 10 windows; this checks the implementation on small cases, in the style of property-based testing [12], and is not part of the proof.

Running time. Fig. 2(a) shows the time following $c\, n \log_2 n$ over a $4096\times$ range of n, from 0.1 ms to 1.6 s. In Fig. 2(b) the ratio is flat, between 10.6 and 12.3 ns, up to $n = 2^{18}$, then rises to 17.3 ns at $n = 2^{22}$. The scan alone (Fig. 2(c)) costs 20 to 25 ns per window up to $n = 2^{17}$, then up to 117 ns at $n = 2^{22}$. Both rises start around where the input, a list of Python tuples, no longer fits in the processor's cache, and they are consistent with cache misses [13]: after sorting, consecutive windows lie far apart in memory. Copying the sorted windows into freshly allocated tuples cut the scan at $n = 2^{22}$ from 118 to 70 ns per window. The output size grew linearly, one refresh day per about 39 windows. Within the cache, the measurements match Theorem 2: $n \log n$ with the sort, linear without it.

III. DIVIDE AND CONQUER: CAPPED MAP SEARCH

A. Problem

Airbnb's map search returns the listings inside the visible map window, but never more than a fixed number of them. When a window holds more, the rest are cut off and cannot be reached from that window. A market study of a city needs *every* listing, so a scraper must choose map windows small enough that none is truncated, using as few searches as possible. This problem came up while scraping Miami, FL and Rincón, PR [14]; a single city-wide window returned only a small fraction of the listings.

B. Abstraction

Reported coordinates have fixed precision (Airbnb gives at most five decimals), so we measure locations in grid units and model the map as $\mathbb{Z}^2$. Let P be a set of n *listings*, each with a location in $\mathbb{Z}^2$. Several listings may share a location; let m be the largest number sharing one. A *window* is a half-open rectangle $r = [x_0, x_1) \times [y_0, y_1)$ with integer corners; its *side* is $\max(x_1 - x_0,\ y_1 - y_0)$, and P_r denotes the listings located in r. The search region R is a window with $P_R = P$ and side W.

The search service is an oracle with cap $k \geq 1$:

$$Q_k(r) = \begin{cases} P_r & \text{if } |P_r| \leq k, \\ \text{OVERFLOW} & \text{otherwise.} \end{cases} \tag{2}$$

The problem is to compute P from R and Q_k using as few oracle calls (web requests) as possible.

Splitting a window at $x_m = \lfloor (x_0 + x_1)/2 \rfloor$ and $y_m = \lfloor (y_0 + y_1)/2 \rfloor$ gives four *quadrants* $r_1, \ldots, r_4$. Splitting recursively yields a *quadtree* [15], [16] whose leaves are the windows actually read.

C. Algorithm

COLLECT (Fig. 4) queries a window. If it is not truncated, its listings are the answer. Otherwise the window is divided into four quadrants, each is solved recursively, and the answers are united. The full answer is COLLECT(R).

D. Correctness

We first show that any answer COLLECT produces is right (Theorem 3), then that it always produces one when $m \leq k$ (Theorem 4).

Lemma 3*: Every location in r lies in exactly one quadrant of r, and no location outside r lies in any quadrant.*

Proof: Since $x_0 \leq x_1$ we have $x_0 \leq x_m \leq x_1$, and likewise for y_m. For $(x, y) \in r$, exactly one of $x < x_m$ and $x \geq x_m$ holds, and exactly one of $y < y_m$ and $y \geq y_m$; the pair selects exactly one quadrant. A location outside r violates one of the four bounds of r, and every quadrant keeps or tightens each of them. ∎

Because the windows are half-open, a listing on a split line falls in exactly one quadrant, so it is neither lost nor counted twice.

Theorem 3 (partial correctness)*: If* COLLECT(r) *terminates, it returns P_r, every leaf window holds at most k listings, and*

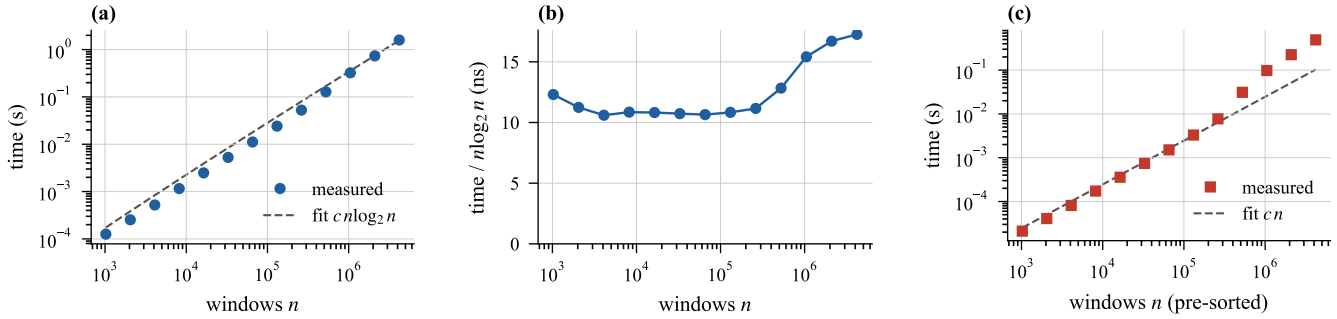


Fig. 2. REFRESH-DAYS on random windows. (a) Running time with the least-squares fit $cn\log_2 n$. (b) Running time divided by $n\log_2 n$; a flat curve confirms the growth rate. (c) The scan alone on pre-sorted windows, with the fit cn over $n \le 2^{17}$.

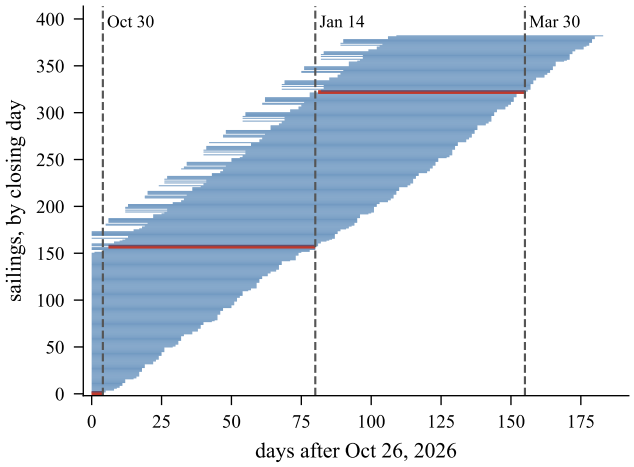


Fig. 3. Quote windows of the 383 real sailings, sorted by closing day. Dashed lines are the three refresh days; the three witnesses are drawn thick and red.

1: **function** COLLECT(r)
2: $A \leftarrow Q_k(r)$
3: **if** $A \neq$ OVERFLOW **then**
4: **return** A {window fits under the cap}
5: **end if**
6: $r_1, \ldots, r_4 \leftarrow$ quadrants of r {divide}
7: **return** $\bigcup_{i=1}^{4}$ COLLECT(r_i) {conquer, combine}

Fig. 4. Enumerating the listings in window r through a capped oracle Q_k.

every location in r lies in exactly one leaf window (and no location outside r in any).

Proof: Strong induction on the height h of the recursion tree. If $h = 0$, the oracle did not overflow, so $|P_r| \le k$ and line 4 returns $Q_k(r) = P_r$; the only leaf is r itself. If $h > 0$, the algorithm recurses on the quadrants $r_1, \ldots, r_4$, whose recursion trees are shorter than h, so by induction COLLECT(r_i) $= P_{r_i}$, the leaves under r_i hold at most k listings, and each location is in exactly $[p \in r_i]$ of them. By Lemma 3 each listing of P_r lies in exactly one r_i, so $\bigcup_i P_{r_i} = P_r$; and the number of leaves containing a location p is $\sum_i [p \in r_i] = [p \in r]$, again by Lemma 3. ■

Lemma 4*: A window of side at most* 1 *contains at most one location. Hence if $m \le k$, such a window holds at most k listings and is never split.*

Proof: If $x_0 \le x < x_1 \le x_0 + 1$ then $x = x_0$, and likewise $y = y_0$, so P_r consists of listings at (x_0, y_0), of which there are at most $m \le k$. ■

Lemma 5*: If r has side at most $2B$, each quadrant of r has side at most B.*

Proof: With $S = x_1 - x_0 \le 2B$, the two horizontal halves have widths $x_m - x_0 = \lfloor S/2 \rfloor$ and $x_1 - x_m = \lceil S/2 \rceil$, both at most B; the same holds vertically. ■

Theorem 4 (termination and depth)*: If $m \le k$ and r has side at most 2^D, then* COLLECT(r) *terminates with recursion depth at most D. In particular* COLLECT(R) *has height at most* $\lceil \log_2 W \rceil$.

Proof: Induction on D. If $D = 0$, the side is at most 1, so by Lemma 4 the oracle answers and the recursion stops at depth 0. For $D + 1$: if the oracle answers, we are done; otherwise the quadrants have side at most 2^D by Lemma 5, so by induction each recursive call terminates within depth D, and COLLECT(r) within depth $D + 1$. ■

Corollary 1 (total correctness)*: If $m \le k$,* COLLECT(R) *terminates within depth* $\lceil \log_2 W \rceil$ *and returns P; its leaf windows partition R and each holds at most k listings.*

Proof: Theorem 4 gives termination, Theorem 3 the rest. ■

The condition $m \le k$ is also necessary. If more than k listings share a location, every window containing it overflows, and no strategy using Q_k can reveal them.



Fig. 5. Scaling with n at $k = 50$, for uniform and clustered listings. (a) Running time with the least-squares fit $c\,n\log_2(n/k)$ ($R^2 = 0.995$ uniform, 0.996 clustered). (b) Running time divided by $n\log_2(n/k)$; a flat curve confirms the growth rate. (c) Requests issued, against the lower bound n/k.

Every statement in this subsection is machine-checked in Lean 4 [9] (Appendix B), under the following correspondence:

Lemma 3	`quadrants_partition`
Theorem 3	`collect_correct`
Lemma 4	`unit_window_fits`
Lemma 5	`quadrants_side`
Theorem 4	`collect_terminates`
Corollary 1	`collect_total`

Lean accepts a recursive definition only with a termination argument, so its COLLECT takes the remaining depth budget as an explicit parameter and returns $\perp$ if the budget runs out. Theorem 4 says that the budget $\lceil \log_2 W \rceil$ never runs out. The closest existing formalizations, region quadtrees in Isabelle [17], [18] and k-d trees in Isabelle and Coq [19], [20], obtain termination from a fixed grid exponent or from the point count shrinking at each median split. Neither covers a tree that splits space over arbitrary points.

E. Running Time

Let N be the number of oracle calls and T the total work when a node with $|P_r|$ listings spends $\Theta(|P_r| + 1)$ time returning or distributing them. Let h be the tree height.

Windows at one depth are disjoint (Lemma 3), so each level holds at most n listings and at most $\lfloor n/(k+1) \rfloor$ split windows, each holding at least $k+1$ listings. Internal nodes sit at depths $0, \ldots, h-1$ and have four children each, so

$$\lceil n/k \rceil \le N \le 1 + 4h\left\lfloor \frac{n}{k+1} \right\rfloor, \qquad T = O\big(n(h+1)\big). \quad (3)$$

The lower bound holds because the leaves cover P with at most k listings each. By Theorem 4, $h \le \lceil \log_2 W \rceil$, so $T = O(n \log W)$: the cost is governed by the coordinate resolution, not by n alone.

In the balanced case, where each split sends about $n/4$ listings to each quadrant (true in expectation for uniformly random locations),

$$T(n) = 4\,T(n/4) + \Theta(n) \quad (n > k), \qquad (4)$$

with $T(n) = \Theta(1)$ for $n \le k$. With $a = b = 4$ and $f(n) = \Theta(n) = \Theta(n^{\log_b a})$, case 2 of the Master Theorem [21] gives $T(n) = \Theta(n \log n)$. Unrolling the recursion shows the dependence on k: the tree is complete with $L = \lceil \log_4(n/k) \rceil$ levels below the root, each costing $\Theta(n)$, so

$$T(n) = \Theta\left(n \log \frac{n}{k}\right), \qquad (5)$$

$$N = \sum_{i=0}^{L} 4^i = \frac{4^{L+1} - 1}{3} = \Theta\left(\frac{n}{k}\right), \qquad (6)$$

within a constant factor of the n/k requests any strategy must make.

F. In the Language of the Problem

Search the whole city in one window. If the results fit on the pages the site shows, record them. Otherwise cut the window into four equal tiles and repeat on each. A listing exactly on a cut goes to the tile right of or above it, so it is recorded once. Dense neighborhoods are zoomed into repeatedly, while open water is dismissed after one search. By (6), the number of searches is proportional to n/k, the number of listings divided by the number shown per search, and the zoom never goes deeper than $\lceil \log_2 W \rceil$ levels. With five-decimal coordinates over a one-degree span, $W = 10^5$ and 17 levels suffice. The method fails only at an address with more listings than the cap, such as an apartment building whose units share rounded coordinates. No zoom level separates them, which is why Corollary 1 needs $m \le k$. In Rincón at most 7 listings share a location, so any $k \ge 7$ works.

G. Experiments

Setup. COLLECT was implemented in Python 3.14 (Appendix A) and run on an AMD Ryzen 7 9800X3D. The oracle is simulated. Each node partitions the listings it holds among its quadrants, which is the $\Theta(|P_r|)$ step of (4). Coordinates are double-precision numbers in the unit square, i.e. the grid model at resolution 2^{-52}; the implementation also stops at windows narrower than 10^{-9} and flags them, which never happened. We used $k = 50$ and $n = 2^{10}, \ldots, 2^{19}$ listings, either uniform or drawn from 20 Gaussian clusters ($\sigma = 0.02$). Each size was run five times on fresh random inputs; we report

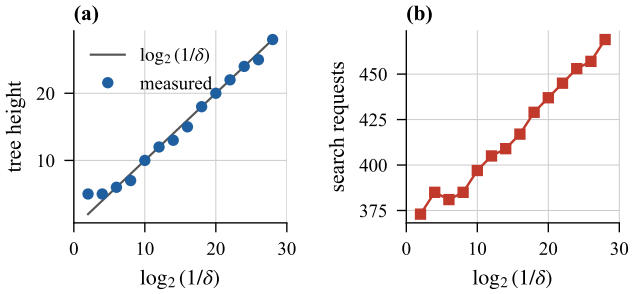


Fig. 6. Effect of packing $k+1$ listings into a square of side δ ($n = 10{,}000$, $k = 50$): (a) tree height, (b) requests.

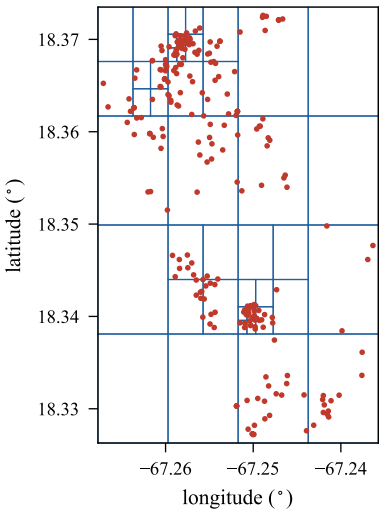


Fig. 7. Windows chosen for 261 listings in Rincón, PR ($k = 20$).

the fastest run with garbage collection disabled, as `timeit` does [11]. Every run was checked to return each listing exactly once.

Running time. Fig. 5(a) shows measured time against the fitted model. Fig. 5(b) divides time by $n \log_2(n/k)$. Over a $512\times$ range of n the ratio stays between 26 and 43 ns (uniform) and between 36 and 48 ns (clustered), confirming $\Theta(n \log(n/k))$. The zig-zag arises because the tree gains a level only when n grows about fourfold, so the true cost rises in steps. The slight rise at the largest sizes is consistent with cache effects. Clustered inputs cost about 20% more because their trees are two levels taller.

Requests. Table I compares requests on uniform inputs with the exact balanced count (6); they agree within 3.1%. Fig. 5(c) shows the count growing linearly, parallel to the lower bound n/k.

Crowding. To test the depth bound, we fixed $n = 10{,}000$ uniform listings and added $k + 1 = 51$ listings inside a square of side $\delta = 2^{-2}, \ldots, 2^{-28}$. Isolating that square requires windows of side about δ, i.e. a grid of $W = 2^e$ units when $\delta = 2^{-e}$. Fig. 6 shows the tree height tracking $\log_2(1/\delta) = \log_2 W$ (Theorem 4) although n is fixed, and each extra level adding about four requests, the $4h$ term of (3).

TABLE I
REQUESTS ON UNIFORM INPUTS ($k = 50$) VS. THE PREDICTION (6)

n	measured	predicted	height
1,024	85	85	3
4,096	333	341	4
16,384	1,337	1,365	5
65,536	5,293	5,461	6
262,144	21,197	21,845	7
524,288	21,917	21,845	8

Real data. Fig. 7 runs the algorithm with $k = 20$ on 261 Airbnb listings from Rincón, PR (scraped March 29, 2023 [14]). It used 57 requests, 43 windows and height 5, within the bounds $14 \le N \le 241$ of (3). The two dense clusters are split repeatedly; the outskirts are covered by a few large windows.

IV. Conclusion

For the cruise-fare tracker, scanning quote windows by closing day and refreshing on the last day of each window not yet served gives the fewest refresh batches, and the windows that forced a refresh prove that no schedule can do better. The algorithm is a sort followed by a linear scan; measured times follow $n \log n$ until the largest inputs, where the cost per window rises, consistent with cache misses. The result rests on the modeling choices stated in Section II: equal-cost batches, one observation per window, and windows defined by the final-payment deadline and an assumed booking cutoff. It minimizes calls to the API, not missed fare changes.

For the capped map search, splitting overflowing windows into quadrants collects every listing with a number of requests within a constant factor of the n/k lower bound when listings are spread evenly, and never recurses deeper than $\lceil \log_2 W \rceil$ levels on a map of W grid units. The one assumption, that no single location holds more than k listings, is necessary for any strategy that only sees capped search results. Measured running times follow the $\Theta(n \log(n/k))$ analysis over three orders of magnitude, request counts match the balanced-case prediction within 3.1%, and the correctness argument is machine-checked.

References

[1] O. Etzioni, R. Tuchinda, C. A. Knoblock, and A. Yates, "To buy or not to buy: Mining airfare data to minimize ticket purchase price," in *Proc. 9th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2003, pp. 119–128.

[2] M. Nottingham and R. T. Fielding, "Additional HTTP status codes," RFC 6585, IETF, https://www.rfc-editor.org/rfc/rfc6585, 2012.

[3] R. Rodriguez, "Cruise finder: self-hosted tracker for Carnival sailings from Florida," https://ryanrodri.com/cruisemaxx, 2026, accessed Oct. 2026.

[4] Carnival Cruise Line, "Deposit and final payment requirements," https://help.carnival.com/app/answers/detail/a_id/481, archived Jul. 23, 2025, https://web.archive.org/web/20250723070445/https://help.carnival.com/app/answers/detail/a_id/481/~/deposit-and-final-payment-requirements; accessed Oct. 2026.

[5] J. Cho and H. Garcia-Molina, "Effective page refresh policies for web crawlers," *ACM Trans. Database Systems*, vol. 28, no. 4, pp. 390–426, 2003.

[6] U. I. Gupta, D. T. Lee, and J. Y.-T. Leung, "Efficient algorithms for interval graphs and circular-arc graphs," *Networks*, vol. 12, no. 4, pp. 459–467, 1982.

[7] J. Kleinberg and É. Tardos, *Algorithm Design*. Pearson/Addison-Wesley, 2006.

[8] M. C. Golumbic, *Algorithmic Graph Theory and Perfect Graphs*, 2nd ed., ser. Annals of Discrete Mathematics. Elsevier, 2004, vol. 57.

[9] L. de Moura and S. Ullrich, "The Lean 4 theorem prover and programming language," in *Automated Deduction – CADE 28*, ser. LNCS, vol. 12699. Springer, 2021, pp. 625–635.

[10] J. I. Munro and S. Wild, "Nearly-optimal mergesorts: Fast, practical sorting methods that optimally adapt to existing runs," in *26th European Symposium on Algorithms (ESA 2018)*, ser. LIPIcs, vol. 112, 2018, pp. 63:1–63:16.

[11] Python Software Foundation, "timeit — measure execution time of small code snippets," https://docs.python.org/3/library/timeit.html, accessed Oct. 2026.

[12] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. 5th ACM SIGPLAN Int. Conf. Functional Programming (ICFP)*, 2000, pp. 268–279.

[13] U. Drepper, "What every programmer should know about memory," Red Hat, Inc., https://www.akkadia.org/drepper/cpumemory.pdf, 2007.

[14] I. Danial, "airbnb_data: Airbnb listing scraper," https://github.com/IlanDanial/airbnb_data, 2023, accessed Oct. 2026.

[15] R. A. Finkel and J. L. Bentley, "Quad trees: A data structure for retrieval on composite keys," *Acta Informatica*, vol. 4, no. 1, pp. 1–9, 1974.

[16] H. Samet, "The quadtree and related hierarchical data structures," *ACM Computing Surveys*, vol. 16, no. 2, pp. 187–260, 1984.

[17] T. Nipkow, "Region quadtrees verified," in *Taming the Infinities of Concurrency: Essays Dedicated to Javier Esparza on the Occasion of His 60th Birthday*, ser. LNCS, S. Kiefer, J. Křetínský, and A. Kučera, Eds. Springer, 2024, vol. 14660, pp. 243–254.

[18] ——, "Region quadtrees," Archive of Formal Proofs, https://isa-afp.org/entries/Region_Quadtrees.html, 2024.

[19] M. Rau, "Multidimensional binary search trees," Archive of Formal Proofs, https://isa-afp.org/entries/KD_Tree.html, 2019.

[20] N. A. Hamid, "(nearest) neighbors you can rely on: Formally verified k-d tree construction and search in Coq," arXiv:2311.10965, 2023.

[21] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. MIT Press, 2022.

APPENDIX A
DIVIDE-AND-CONQUER CODE

The algorithm (`quadtree.py`) and the measurements behind Figs. 5 and 6 and Table I (`experiments.py`). The plotting script, the unit tests, the Rincón data file and the raw measurements are in the project repository.[1]

Listing 1. quadtree.py

```
"""Adaptive quadtree decomposition for capped map search (divide and conquer).

A map-search service (e.g. Airbnb's map view) returns the listings inside a
rectangular window only when the window holds at most k of them; otherwise it
truncates the results. To enumerate every listing in a region we split the
window into four quadrants and recurse until every window fits under the cap.

Abstraction: P is a set of n points in a rectangle R. The leaves of the
recursion form a partition of R into half-open rectangles, each holding at
most k points of P, and the union of their points is exactly P.
"""

import math
from dataclasses import dataclass, field


@dataclass(frozen=True)
class Rect:
    """Half-open rectangle [x0, x1) x [y0, y1).

    Half-open bounds guarantee that a point on a split line belongs to exactly
    one quadrant, so no listing is reported twice or skipped.
    """

    x0: float
    y0: float
    x1: float
    y1: float

    def contains(self, p):
        return self.x0 <= p[0] < self.x1 and self.y0 <= p[1] < self.y1

    def area(self):
        return (self.x1 - self.x0) * (self.y1 - self.y0)

    def quadrants(self, xm, ym):
        # Order matches the bucket index (x >= xm) + 2 * (y >= ym) in _collect.
        return (
            Rect(self.x0, self.y0, xm, ym),  # 0: lower-left
            Rect(xm, self.y0, self.x1, ym),  # 1: lower-right
            Rect(self.x0, ym, xm, self.y1),  # 2: upper-left
            Rect(xm, ym, self.x1, self.y1),  # 3: upper-right
        )


@dataclass
class Result:
    points: list = field(default_factory=list)  # every listing found
    leaves: list = field(default_factory=list)  # windows that fit under the cap
    queries: int = 0  # search requests issued (one per recursion node)
    max_depth: int = 0  # height of the quadtree
    unresolved: int = 0  # windows that hit min_width while still over the cap


def bounding_rect(points):
    """Smallest half-open rectangle containing every point.

    The upper bounds are nudged to the next float so the points with the
    largest x or y still fall inside the half-open box.
    """
    xs = [p[0] for p in points]
    ys = [p[1] for p in points]
    return Rect(min(xs), min(ys),
                math.nextafter(max(xs), math.inf), math.nextafter(max(ys), math.inf))


def collect(rect, points, k, min_width=1e-9):
    """Return every point in `rect` using windows of at most k points.

    `points` is the ground truth the simulated search service holds for
    `rect`; the algorithm only learns it through the capped query on each
    window. If more than k points share (nearly) the same location no window
```

[1] https://github.com/IlanDanial/P1Alg

```
    can isolate them, so recursion stops once a window is narrower than
    `min_width` and the window is counted as unresolved.
    """
    if k < 1:
        raise ValueError("k must be at least 1")
    result = Result()
    _collect(rect, points, k, min_width, 0, result)
    return result


def _collect(rect, points, k, min_width, depth, result):
    result.queries += 1
    result.max_depth = max(result.max_depth, depth)

    # Base case: the service shows the whole window.
    if len(points) <= k:
        result.points.extend(points)
        result.leaves.append(rect)
        return

    # Cannot zoom any further: keep the truncated page the service shows.
    if rect.x1 - rect.x0 <= min_width or rect.y1 - rect.y0 <= min_width:
        result.points.extend(points[:k])
        result.leaves.append(rect)
        result.unresolved += 1
        return

    # Divide: split the window at its midpoint into four quadrants.
    xm = (rect.x0 + rect.x1) / 2
    ym = (rect.y0 + rect.y1) / 2
    buckets = ([], [], [], [])
    for p in points:
        buckets[(p[0] >= xm) + 2 * (p[1] >= ym)].append(p)

    # Conquer each quadrant; combining is just appending to the shared result.
    for quad, bucket in zip(rect.quadrants(xm, ym), buckets):
        _collect(quad, bucket, k, min_width, depth + 1, result)
```

Listing 2. experiments.py

```
"""Running-time experiments for quadtree.collect.

Three experiments, each written to results/*.csv:
  scaling_uniform.csv, scaling_clustered.csv  time, requests and tree height
                                              as n grows (balanced case)
  depth_vs_delta.csv                          tree height as k+1 listings are
                                              packed ever closer together
plots.py turns the CSV files into the figures in the report.

Run: python experiments.py   (about a minute)
"""

import csv
import gc
import math
import random
import statistics
import time
from collections import Counter
from pathlib import Path

from quadtree import Rect, collect

RESULTS = Path(__file__).parent / "results"
K = 50  # listings a search shows before truncating (a few result pages)
SIZES = [2**e for e in range(10, 20)]  # 1,024 .. 524,288 listings
TRIALS = 5  # random inputs per size
UNIT = Rect(0.0, 0.0, 1.0, 1.0)


# -- inputs -------------------------------------------------------------

def uniform(n, rng):
    """n listings spread evenly over the unit square."""
    return [(rng.random(), rng.random()) for _ in range(n)]


def clustered(n, rng, centers=20, spread=0.02):
    """n listings concentrated around a few neighborhoods (Gaussian clusters)."""
    cs = [(rng.random(), rng.random()) for _ in range(centers)]
    pts = []
    while len(pts) < n:
        cx, cy = rng.choice(cs)
```

```
        x, y = rng.gauss(cx, spread), rng.gauss(cy, spread)
        if 0.0 <= x < 1.0 and 0.0 <= y < 1.0:
            pts.append((x, y))
    return pts


# -- predictions from the analysis -------------------------------------------

def model(n):
    """Predicted work n log2(n/k), from T(n) = 4T(n/4) + Theta(n)."""
    return n * math.log2(n / K)


def balanced_queries(n):
    """Requests when every split is even: a complete 4-ary tree of L+1 levels."""
    levels = 0
    while K * 4**levels < n:
        levels += 1
    return (4 ** (levels + 1) - 1) // 3


def fit_constant(rows):
    """Least-squares c for time = c * model(n) over the rows, and the fit's R^2."""
    f = [model(r["n"]) for r in rows]
    t = [r["time_s"] for r in rows]
    c = sum(a * b for a, b in zip(f, t)) / sum(a * a for a in f)
    mean = statistics.mean(t)
    ss_res = sum((ti - c * fi) ** 2 for fi, ti in zip(f, t))
    ss_tot = sum((ti - mean) ** 2 for ti in t)
    return c, 1 - ss_res / ss_tot


# -- measurement -------------------------------------------------------------

def time_collect(pts):
    """Time collect() on pts, then check it returned every listing exactly once."""
    # Like timeit, keep the garbage collector from adding pauses to the timing.
    gc.collect()
    gc.disable()
    try:
        start = time.perf_counter()
        result = collect(UNIT, pts, K)
        elapsed = time.perf_counter() - start
    finally:
        gc.enable()
    assert result.unresolved == 0 and Counter(result.points) == Counter(pts)
    return elapsed, result


def run_scaling(name, gen):
    """Time collect() for each n in SIZES on TRIALS fresh inputs from gen.

    Reports the fastest time (the least-disturbed run, as timeit recommends)
    and the median request count and tree height.
    """
    rows = []
    for n in SIZES:
        times, queries, depths = [], [], []
        for t in range(TRIALS):
            pts = gen(n, random.Random(1000 * n + t))
            elapsed, result = time_collect(pts)
            times.append(elapsed)
            queries.append(result.queries)
            depths.append(result.max_depth)
        row = {
            "n": n,
            "time_s": min(times),
            "queries": int(statistics.median(queries)),
            "balanced_queries": balanced_queries(n),
            "max_depth": int(statistics.median(depths)),
        }
        rows.append(row)
        print(f"{name:9s} n={n:7d}  time={row['time_s']:.4f}s  "
              f"queries={row['queries']}  depth={row['max_depth']}")
    write_csv(RESULTS / f"scaling_{name}.csv", rows)
    return rows


def run_depth(n=10_000, seed=7):
    """Pack k+1 listings (a 'building') into a square of side delta = 2^-e.

    n stays fixed, so any growth in height and requests comes from the
    crowding alone. The building is placed away from the map's edges so the
```

```
    quadrants that isolate it are not clipped by the border.
    """
    rows = []
    for e in range(2, 29, 2):  # stays above collect's min_width of 1e-9 ~ 2^-30
        delta = 2.0**-e
        rng = random.Random(seed)
        pts = uniform(n - (K + 1), rng)
        cx, cy = 0.3 + 0.4 * rng.random(), 0.3 + 0.4 * rng.random()
        pts += [(cx + delta * rng.random(), cy + delta * rng.random()) for _ in range(K + 1)]
        elapsed, result = time_collect(pts)
        rows.append({"log2_inv_delta": e, "time_s": elapsed,
                     "queries": result.queries, "max_depth": result.max_depth})
        print(f"depth     delta=2^-{e:<2d} queries={result.queries}  depth={result.max_depth}")
    write_csv(RESULTS / "depth_vs_delta.csv", rows)
    return rows


def write_csv(path, rows):
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)


if __name__ == "__main__":
    RESULTS.mkdir(exist_ok=True)
    run_scaling("uniform", uniform)
    run_scaling("clustered", clustered)
    run_depth()
```

APPENDIX B
MACHINE-CHECKED PROOF

Section III-D in Lean 4.35, using only the core library: Lemma 3 (`quadrants_partition`), Theorem 3 (`collect_correct`), Lemma 4 (`unit_window_fits`), Lemma 5 (`quadrants_side`), Theorem 4 (`collect_terminates`) and Corollary 1 (`collect_total`). `lake build` in `divide_and_conquer/proof` checks it; the theorems depend on no axioms beyond Lean's standard three (`propext`, `Classical.choice`, `Quot.sound`).

Listing 3. proof/Proof/Collect.lean

```
/-!
# Capped map search: machine-checked correctness of `collect`

Formalizes Section III-D of the report, statement for statement. Coordinates
are integers: reported coordinates have fixed precision, so the plane is a
grid. Only Lean's core library is used; every arithmetic step is discharged by
`omega`.

| Report      | Lean                    |
|-------------|-------------------------|
| Lemma 3     | `quadrants_partition`   |
| Theorem 3   | `collect_correct`       |
| Lemma 4     | `unit_window_fits`      |
| Lemma 5     | `quadrants_side`        |
| Theorem 4   | `collect_terminates`    |
| Corollary 1 | `collect_total`         |

`collect` carries its remaining depth budget as an explicit argument and
returns `none` if the budget runs out. Theorem 4 shows that with budget
`⌈log₂ W⌉` this never happens when no location holds more than `k` listings.
-/

namespace Quadtree

/-- A half-open window `[x0, x1) × [y0, y1)`. -/
structure Window where
  x0 : Int
  x1 : Int
  y0 : Int
  y1 : Int

/-- The location `p` lies in window `w`. -/
def inWin (p : Int × Int) (w : Window) : Prop :=
  w.x0 ≤ p.1 ∧ p.1 < w.x1 ∧ w.y0 ≤ p.2 ∧ p.2 < w.y1

instance (p : Int × Int) (w : Window) : Decidable (inWin p w) := by
  unfold inWin; infer_instance

/-- The four quadrants of `w`, split at `⌊(x0+x1)/2⌋` and `⌊(y0+y1)/2⌋`. -/
```

```
def q1 (w : Window) : Window := ⟨w.x0, (w.x0 + w.x1) / 2, w.y0, (w.y0 + w.y1) / 2⟩
def q2 (w : Window) : Window := ⟨(w.x0 + w.x1) / 2, w.x1, w.y0, (w.y0 + w.y1) / 2⟩
def q3 (w : Window) : Window := ⟨w.x0, (w.x0 + w.x1) / 2, (w.y0 + w.y1) / 2, w.y1⟩
def q4 (w : Window) : Window := ⟨(w.x0 + w.x1) / 2, w.x1, (w.y0 + w.y1) / 2, w.y1⟩

/-- `1` if `p` lies in `w`, else `0`. -/
def ind (p : Int × Int) (w : Window) : Nat := if inWin p w then 1 else 0

/-! ## Lemma 3: the quadrants partition the window -/

/-- **Lemma 3.** A location in `w` lies in exactly one quadrant; a location
outside `w` lies in none. -/
theorem quadrants_partition (p : Int × Int) (w : Window) :
    ind p (q1 w) + ind p (q2 w) + ind p (q3 w) + ind p (q4 w) = ind p w := by
  -- Decide each of the five memberships, then the remaining goal is linear arithmetic.
  by_cases h0 : inWin p w <;> by_cases h1 : inWin p (q1 w) <;> by_cases h2 : inWin p (q2 w) <;>
    by_cases h3 : inWin p (q3 w) <;> by_cases h4 : inWin p (q4 w) <;>
    simp only [ind, h0, h1, h2, h3, h4, ite_true, ite_false] <;>
    simp only [inWin, q1, q2, q3, q4] at h0 h1 h2 h3 h4 <;> omega

/-- Lemma 3 in its set form: `p` is in `w` iff it is in some quadrant. -/
theorem quadrants_cover (p : Int × Int) (w : Window) :
    inWin p w ↔ inWin p (q1 w) ∨ inWin p (q2 w) ∨ inWin p (q3 w) ∨ inWin p (q4 w) := by
  simp only [inWin, q1, q2, q3, q4]
  omega

/-! ## The algorithm -/

variable {α : Type} (P : List α) (loc : α → Int × Int) (k : Nat)

/-- `P_w`: the listings located in `w`. -/
def listingsIn (w : Window) : List α :=
  P.filter (fun a => decide (inWin (loc a) w))

/-- The capped search: the listings in `w` if there are at most `k`, else overflow. -/
def oracle (w : Window) : Option (List α) :=
  if (listingsIn P loc w).length ≤ k then some (listingsIn P loc w) else none

/-- `Collect` from the report, with an explicit depth budget. Returns the
listings found and the leaf windows, or `none` if the budget runs out. -/
def collect : Nat → Window → Option (List α × List Window)
  | 0, w =>
    match oracle P loc k w with
    | some A => some (A, [w])
    | none => none
  | fuel + 1, w =>
    match oracle P loc k w with
    | some A => some (A, [w])
    | none =>
      match collect fuel (q1 w), collect fuel (q2 w),
            collect fuel (q3 w), collect fuel (q4 w) with
      | some (o1, l1), some (o2, l2), some (o3, l3), some (o4, l4) =>
        some (o1 ++ o2 ++ o3 ++ o4, l1 ++ l2 ++ l3 ++ l4)
      | _, _, _, _ => none

/-- Number of leaves in `leaves` containing `p`. -/
def leafCount (p : Int × Int) (leaves : List Window) : Nat :=
  leaves.countP (fun l => decide (inWin p l))

/-- The three properties Theorem 3 asserts of an answer `(out, leaves)` for `w`:
`out` is exactly `P_w`; every leaf is under the cap; every location in `w` lies
in exactly one leaf and every location outside `w` in none. -/
def Correct (w : Window) (out : List α) (leaves : List Window) : Prop :=
  (∀ a, a ∈ out ↔ a ∈ P ∧ inWin (loc a) w) ∧
  (∀ l ∈ leaves, (listingsIn P loc l).length ≤ k) ∧
  (∀ p, leafCount p leaves = ind p w)

/-! ## Theorem 3: partial correctness -/

theorem mem_listingsIn (a : α) (w : Window) :
    a ∈ listingsIn P loc w ↔ a ∈ P ∧ inWin (loc a) w := by
  simp [listingsIn, List.mem_filter]

theorem oracle_some {w : Window} {A : List α} (h : oracle P loc k w = some A) :
    A = listingsIn P loc w ∧ (listingsIn P loc w).length ≤ k := by
  unfold oracle at h
  split at h
  · cases h; exact ⟨rfl, by assumption⟩
  · cases h

theorem leafCount_singleton (p : Int × Int) (w : Window) : leafCount p [w] = ind p w := by
  simp only [leafCount, ind, List.countP_cons, List.countP_nil, decide_eq_true_eq]
  split <;> rfl
```

```
theorem leafCount_append (p : Int × Int) (l₁ l₂ : List Window) :
    leafCount p (l₁ ++ l₂) = leafCount p l₁ + leafCount p l₂ := by
  simp [leafCount, List.countP_append]

/-- A non-overflowing window is a correct leaf. -/
theorem leaf_correct {w : Window} {A : List α} (h : oracle P loc k w = some A) :
    Correct P loc k w A [w] := by
  obtain ⟨rfl, hk⟩ := oracle_some P loc k h
  refine ⟨fun a => mem_listingsIn P loc a w, ?_, fun p => leafCount_singleton p w⟩
  intro l hl
  simp only [List.mem_singleton] at hl
  subst hl; exact hk

/-- **Theorem 3.** Whenever `collect` returns an answer, the answer is correct. -/
theorem collect_correct (fuel : Nat) :
    ∀ (w : Window) (out : List α) (leaves : List Window),
      collect P loc k fuel w = some (out, leaves) → Correct P loc k w out leaves := by
  induction fuel with
  | zero =>
    intro w out leaves h
    unfold collect at h
    split at h
    · cases h; exact leaf_correct P loc k (by assumption)
    · cases h
  | succ fuel ih =>
    intro w out leaves h
    unfold collect at h
    split at h
    · cases h; exact leaf_correct P loc k (by assumption)
    · split at h
      · rename_i o1 l1 o2 l2 o3 l3 o4 l4 h1 h2 h3 h4
        cases h
        obtain ⟨m1, c1, v1⟩ := ih _ _ _ h1
        obtain ⟨m2, c2, v2⟩ := ih _ _ _ h2
        obtain ⟨m3, c3, v3⟩ := ih _ _ _ h3
        obtain ⟨m4, c4, v4⟩ := ih _ _ _ h4
        refine ⟨?_, ?_, ?_⟩
        · -- out = P_w: a listing is in some quadrant's output iff it is in w (Lemma 3).
          intro a
          rw [quadrants_cover]
          simp only [List.mem_append, m1, m2, m3, m4, and_or_left, or_assoc]
        · intro l hl
          simp only [List.mem_append] at hl
          rcases hl with ((hl | hl) | hl) | hl
          · exact c1 l hl
          · exact c2 l hl
          · exact c3 l hl
          · exact c4 l hl
        · intro p
          rw [leafCount_append, leafCount_append, leafCount_append, v1, v2, v3, v4]
          exact quadrants_partition p w
      · cases h

/-! ## Lemma 4, Lemma 5, Theorem 4: the depth budget `⌈log₂ W⌉` suffices -/

/-- `m ≤ k`: no single location holds more than `k` listings. -/
def CapOK : Prop := ∀ p : Int × Int, (P.filter (fun a => decide (loc a = p))).length ≤ k

/-- A filter by a stronger predicate is no longer than a filter by a weaker one. -/
theorem length_filter_le_of_imp (l : List α) (p q : α → Bool) (h : ∀ a, p a = true → q a = true) :
    (l.filter p).length ≤ (l.filter q).length := by
  induction l with
  | nil => simp
  | cons a l ih =>
    simp only [List.filter_cons]
    cases hp : p a with
    | true =>
      have hq := h a hp
      simp only [hq, ite_true, List.length_cons]
      omega
    | false =>
      simp only [Bool.false_eq_true, ite_false]
      split
      · simp only [List.length_cons]; omega
      · exact ih

/-- **Lemma 4.** A window of side at most 1 holds listings of a single
location, so under `m ≤ k` it is never split. -/
theorem unit_window_fits (hcap : CapOK P loc k) (w : Window)
    (hx : w.x1 - w.x0 ≤ 1) (hy : w.y1 - w.y0 ≤ 1) :
    (listingsIn P loc w).length ≤ k := by
  refine Nat.le_trans (length_filter_le_of_imp P _ _ ?_) (hcap (w.x0, w.y0))
```

```
  intro a ha
  simp only [decide_eq_true_eq, inWin, Prod.ext_iff] at ha ⊢
  omega

/-- **Lemma 5.** The quadrants of a window of side at most `2B` have side at most `B`. -/
theorem quadrants_side (w : Window) (B : Int)
    (hx : w.x1 - w.x0 ≤ 2 * B) (hy : w.y1 - w.y0 ≤ 2 * B) :
    ((q1 w).x1 - (q1 w).x0 ≤ B ∧ (q1 w).y1 - (q1 w).y0 ≤ B) ∧
    ((q2 w).x1 - (q2 w).x0 ≤ B ∧ (q2 w).y1 - (q2 w).y0 ≤ B) ∧
    ((q3 w).x1 - (q3 w).x0 ≤ B ∧ (q3 w).y1 - (q3 w).y0 ≤ B) ∧
    ((q4 w).x1 - (q4 w).x0 ≤ B ∧ (q4 w).y1 - (q4 w).y0 ≤ B) := by
  simp only [q1, q2, q3, q4]
  omega

/-- **Theorem 4.** Under `m ≤ k`, on a window of side at most `2^D`, `collect`
with depth budget `D` returns an answer. -/
theorem collect_terminates (hcap : CapOK P loc k) (D : Nat) :
    ∀ w : Window, w.x1 - w.x0 ≤ (2 : Int) ^ D → w.y1 - w.y0 ≤ (2 : Int) ^ D →
      ∃ r, collect P loc k D w = some r := by
  induction D with
  | zero =>
    intro w hx hy
    simp only [Int.pow_zero] at hx hy
    have hfits := unit_window_fits P loc k hcap w hx hy
    refine ⟨(listingsIn P loc w, [w]), ?_⟩
    simp [collect, oracle, hfits]
  | succ D ih =>
    intro w hx hy
    rw [Int.pow_succ, Int.mul_comm] at hx hy
    obtain ⟨⟨a1, b1⟩, ⟨a2, b2⟩, ⟨a3, b3⟩, ⟨a4, b4⟩⟩ := quadrants_side w _ hx hy
    unfold collect
    split
    · exact ⟨_, rfl⟩
    · obtain ⟨⟨o1, l1⟩, h1⟩ := ih (q1 w) a1 b1
      obtain ⟨⟨o2, l2⟩, h2⟩ := ih (q2 w) a2 b2
      obtain ⟨⟨o3, l3⟩, h3⟩ := ih (q3 w) a3 b3
      obtain ⟨⟨o4, l4⟩, h4⟩ := ih (q4 w) a4 b4
      rw [h1, h2, h3, h4]
      exact ⟨_, rfl⟩

/-- **Corollary 1 (total correctness).** Under `m ≤ k`, on a window of side at
most `2^D`, `collect` with depth budget `D` returns a correct answer. -/
theorem collect_total (hcap : CapOK P loc k) (D : Nat) (w : Window)
    (hx : w.x1 - w.x0 ≤ (2 : Int) ^ D) (hy : w.y1 - w.y0 ≤ (2 : Int) ^ D) :
    ∃ out leaves, collect P loc k D w = some (out, leaves) ∧ Correct P loc k w out leaves := by
  obtain ⟨⟨out, leaves⟩, h⟩ := collect_terminates P loc k hcap D w hx hy
  exact ⟨out, leaves, h, collect_correct P loc k D w out leaves h⟩

end Quadtree
```

APPENDIX C
LLM USAGE: DIVIDE-AND-CONQUER PART

Tool. Claude Code (Anthropic's command-line coding agent), model Claude Opus 5.5 (`claude-opus-5-5`), October 2026. The agent read the assignment, read the scraper repository [14], searched the web for the IEEE template, wrote and ran the code, and drafted this section. All proofs, numbers, and citations were checked by the authors.

Prompts (verbatim).

1) "DO you understand the project I need to do also this is a group project I am doing the divide an conquer part"
2) "My Idea for the problem is a problem I have solved before with divide and conquer https://github.com/IlanDanial/airbnb_data I split the map over and over, each time checking the amount of properties in the split and if it was under I took those properties and added it to the list since you can only see so many properties at a time"
3) "yes start the implementation, we're using IEEE search online for proper template"
4) "yes write the LaTeX section"
5) "This is really ugly" followed by "My bigger issue is that the explanations are very longwinded for no reason and the formatting of the page is ugly its hard to read as well"
6) "I have an issue with some of this read the paper be critical how do you think of the formatting", then "fix all of them at once" and "also do we need the full code in the appendix or can we reference a repository"
7) "would it be worth it to look at the code and refactor it and the comments in the code", then "go"
8) "Isnt there a code languge that is built to prove proofs can we use that to prove correctness", then "I want to go for it also I am sure there are online proofs of this exact same thing in the same language so it should take less than an hour"
9) "so your telling me that there was no problem with proving the whole thing end to end"
10) "actually search online for the hard part dont just tell me it does not exist"

11) “do both, go for the full proof, or is there a solution we can do that would be easier to prove and solve the problem all the same”
12) “why would anything in the paper not exactly match the proof we did with elon”, then “we need to have the paper be matching hte lean”
13) “can we get rid of common AI’sms like mdashes list a couple more ai isms”
14) Two screenshots of the PDF with “why is this structure so odd”, “things are so packed together that it is unreadable, also the – looks like m dashes”, a request that only the partner’s remaining work be marked as to do, and the author’s affiliation details.
15) “whats left for me to do”, “according to the instructions it seems like it is asking me to list everything in the appendix”, and the decision to export the transcript into the repository rather than typeset it here.

Full transcript. The complete session, exported with Claude Code’s `/export` command, is in the repository at `divide_and_conquer/appendix/transcript.md` (commit `cba125c`), with every prompt and response in order. Intermediate versions of the code, figures and this report are in the repository’s commit history.

Results. (1) A summary of the assignment and four candidate divide-and-conquer problems. (2) An analysis of the scraper’s `subdivide` recursion as an adaptive quadtree, identifying the termination issue for co-located listings and the double-counting issue on window borders. (3) `quadtree.py`, `test_quadtree.py`, `experiments.py`, and the figures; an intermediate run stopped when the minimum-width guard was reached, after which the crowding experiment was limited to $\delta \geq 2^{-28}$, and garbage collection was disabled during timing. (4) A first draft of Section III and these appendices, with five separate figures in matplotlib’s default style. (5) The draft was shortened by about a third, the five figures were restyled to match the paper’s typeface and merged into three, and the page layout was fixed. (6) A list of nine formatting defects (an equation label pushed to its own line, a wrapped pseudocode line, mixed paragraph styles, inconsistent equation citations, a legend mismatch), all then fixed; the unit tests were removed from this appendix in favor of the repository link. (7) The plotting code was split out of `experiments.py` into `plots.py`, and docstrings and comments were added. (8) An assessment of proof assistants (Lean 4, Coq, Isabelle, Dafny) and of which parts of the proof are feasible to formalize; then a first Lean formalization of Lemma 3 and Theorem 3 only, with COLLECT given a fuel bound, which compiled on the first build. (9) A clarification that this was partial correctness, not an end-to-end proof, with three further gaps listed (integer midpoints, the oracle not being a black box, no duplicate-freeness proof). (10) A web search that found the Isabelle and Coq formalizations cited in Section III-D, none covering separation-based termination. The assistant’s earlier claim that no related formalization existed was withdrawn. (11) The observation that finite-precision coordinates make the problem a grid problem, on which Theorem 4 is provable by induction on the depth; `collect_terminates` and `collect_total` were then added, needing one round of fixes to core-library lemma names before compiling. (12) A table of the remaining differences between the paper’s model (real coordinates, exact midpoints, leaves asserted disjoint) and the Lean model; the paper’s abstraction and Section III-D were then rewritten on the integer grid to match the Lean file statement for statement, and leaf disjointness was added to the Lean proof (`quadrants_partition` in counting form). (13) A scan of the report for colon-spliced sentences, filler adverbs and similar tics, which were removed; no em dashes were present. (14) Theorem spacing via `amsthm`, numeric ranges rewritten in words, the front matter and introduction and conclusion for this half written, and the partner’s remaining items marked in red. (15) The advice that the full transcript must be included rather than attached, then the pointer above once the transcript was exported.

APPENDIX D
GREEDY CODE

The algorithm (`refresh_days.py`) and the measurements behind Fig. 2 (`experiments.py`). The unit tests (`test_refresh_days.py`), the download script (`fetch_sailings.py`), the plotting script, the sailing data and the raw measurements are in the `greedy/` folder of the project repository.[2]

Listing 4. refresh_days.py

```
"""Fewest price-refresh batches that observe every quote window (greedy).

Carnival's cruise search is an unofficial, rate-limited endpoint, so a fare
tracker should call it as rarely as it can. One refresh batch queries every
configured departure month and follows every page, so it observes the current
price of every listed sailing. A quote request asks for at least one fresh
observation of its sailing somewhere in a window of days; the tracker must
satisfy every request with as few batches as possible.

Abstraction: n integer intervals [a_i, b_i]. Choose a set T of days such that
every interval contains a day of T, with |T| minimum (minimum point cover of
intervals, also called interval stabbing).
"""
```

[2]https://github.com/IlanDanial/P1Alg

```
FINAL_PAYMENT_SHORT = 76  # days before departure, cruises of 5 nights or fewer
FINAL_PAYMENT_LONG = 91  # days before departure, cruises of 6 nights or more


def final_payment_days(nights):
    """Carnival's final-payment deadline, in days before departure."""
    return FINAL_PAYMENT_SHORT if nights <= 5 else FINAL_PAYMENT_LONG


def windows(sailings, today, c=2):
    """Quote windows for `sailings` as (a, b) day offsets from `today`.

    Each row has `departure` (a date) and `nights` (an int). The window runs
    from the final-payment deadline to booking close, c days before departure.
    Windows that closed before today are dropped; open ones start no earlier
    than today (offset 0), since a refresh cannot happen in the past.
    """
    out = []
    for s in sailings:
        d = (s["departure"] - today).days
        a, b = d - final_payment_days(s["nights"]), d - c
        if a > b:
            raise ValueError(f"empty window for {s}: c={c} is too large")
        if b >= 0:
            out.append((max(a, 0), b))
    return out


def refresh_days(intervals):
    """Return (days, witnesses): a minimum set of days hitting every interval.

    Scan the intervals by closing day. Whenever one is not yet hit, refresh on
    its last day, which hits it and as many later-closing intervals as any
    single day could. The intervals that forced a refresh are the witnesses;
    they are pairwise disjoint, which is why no smaller set of days exists.
    """
    for a, b in intervals:
        if a > b:
            raise ValueError(f"interval ({a}, {b}) has a > b")
    days, witnesses = [], []
    last = None  # most recent refresh day; None plays the role of -infinity
    for a, b in sorted(intervals, key=lambda w: w[1]):
        if last is None or a > last:
            days.append(b)
            witnesses.append((a, b))
            last = b
    return days, witnesses


def refresh_days_sorted(intervals):
    """The scan alone, for intervals already sorted by closing day (O(n))."""
    days, last = [], None
    for a, b in intervals:
        if last is None or a > last:
            days.append(b)
            last = b
    return days
```

Listing 5. experiments.py

```
"""Running-time experiments for refresh_days.

Two experiments, each written to results/*.csv:
  scaling.csv    full algorithm (sort + scan) as n grows
  scan_only.csv  the scan alone on windows already sorted by closing day
plots.py turns the CSV files into the figures in the report.

Run: python experiments.py   (a few minutes)
"""

import bisect
import csv
import gc
import math
import random
import statistics
import time
from pathlib import Path

from fetch_sailings import C, read_sailings
from refresh_days import final_payment_days, refresh_days, refresh_days_sorted
```

```
RESULTS = Path(__file__).parent / "results"
SIZES = [2**e for e in range(10, 23)]  # 1,024 .. 4,194,304 windows
TRIALS = 5  # random inputs per size


# -- inputs ------------------------------------------------------------

def nights_mix():
    """Cruise lengths of the real sailings, sampled to give realistic windows."""
    return [s["nights"] for s in read_sailings()]


def random_windows(n, rng, mix):
    """n quote windows for sailings departing uniformly over 2n days.

    The horizon grows with n so closing days stay mostly distinct; a fixed
    horizon would make most windows tie and measure something else.
    """
    out = []
    for _ in range(n):
        d = rng.randrange(2 * n)
        out.append((d - final_payment_days(rng.choice(mix)), d - C))
    return out


# -- predictions from the analysis -------------------------------------

def model(n):
    """Predicted work n log2 n for the comparison sort (Theorem 2)."""
    return n * math.log2(n)


def fit_constant(rows, f=model):
    """Least-squares c for time = c * f(n) over the rows, and the fit's R^2."""
    fs = [f(r["n"]) for r in rows]
    t = [r["time_s"] for r in rows]
    c = sum(a * b for a, b in zip(fs, t)) / sum(a * a for a in fs)
    mean = statistics.mean(t)
    ss_res = sum((ti - c * fi) ** 2 for fi, ti in zip(fs, t))
    ss_tot = sum((ti - mean) ** 2 for ti in t)
    return c, 1 - ss_res / ss_tot


# -- measurement -------------------------------------------------------

def timed(fn, arg):
    """Time fn(arg) with the garbage collector off, as timeit does."""
    gc.collect()
    gc.disable()
    try:
        start = time.perf_counter()
        out = fn(arg)
        elapsed = time.perf_counter() - start
    finally:
        gc.enable()
    return elapsed, out


def check(iv, days, witnesses):
    """Every window holds a chosen day (Lemma 1); one witness per day (Lemma 2)."""
    assert len(witnesses) == len(days)
    for a, b in iv:
        i = bisect.bisect_left(days, a)  # days come out increasing
        assert i < len(days) and days[i] <= b
    for (_, b1), (a2, _) in zip(witnesses, witnesses[1:]):
        assert b1 < a2


def run(mix):
    """Time refresh_days and the scan alone for each n on TRIALS fresh inputs.

    Reports the fastest time of each (the least-disturbed run, as timeit
    recommends) and the median number of refresh days.
    """
    full, scan = [], []
    for n in SIZES:
        t_full, t_scan, ms = [], [], []
        for t in range(TRIALS):
            iv = random_windows(n, random.Random(1000 * n + t), mix)
            elapsed, (days, witnesses) = timed(refresh_days, iv)
            check(iv, days, witnesses)
            t_full.append(elapsed)
            ms.append(len(days))
```

```
            iv.sort(key=lambda w: w[1])
            elapsed, sdays = timed(refresh_days_sorted, iv)
            assert sdays == days
            t_scan.append(elapsed)
        m = int(statistics.median(ms))
        full.append({"n": n, "time_s": min(t_full), "refresh_days": m})
        scan.append({"n": n, "time_s": min(t_scan), "refresh_days": m})
        print(f"n={n:8d}  full={min(t_full):.4f}s  scan={min(t_scan):.4f}s  m={m}")
    write_csv(RESULTS / "scaling.csv", full)
    write_csv(RESULTS / "scan_only.csv", scan)


def write_csv(path, rows):
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0]))
        w.writeheader()
        w.writerows(rows)


if __name__ == "__main__":
    RESULTS.mkdir(exist_ok=True)
    run(nights_mix())
```

APPENDIX E
GREEDY MACHINE-CHECKED PROOF

Lemma 1 (`refresh_covers`), Lemma 2 (`witnesses_pairwise_disjoint`) and Theorem 1 (`refresh_optimal`) in Lean 4.35, using only the core library. `lake build` in `greedy/proof` checks it; the theorems depend on no axioms beyond Lean's standard three (`propext`, `Classical.choice`, `Quot.sound`).

Listing 6. proof/Proof/Refresh.lean

```
/-!
# Fewest refresh days: machine-checked correctness of `Refresh-Days`

Formalizes Section II-D of the report. Days are integers and a window is a
pair `(a, b)` with `a ≤ b`, meaning the days `a, ..., b`. Only Lean's core
library is used; arithmetic steps are discharged by `omega`.

| Report    | Lean                           |
|-----------|--------------------------------|
| Lemma 1   | `refresh_covers`               |
| Lemma 2   | `witnesses_pairwise_disjoint`  |
| Theorem 1 | `refresh_optimal`              |

`refresh` is the scan of Fig. 1 on a list already sorted by closing day; it
returns the witnesses, and the refresh days are their closing days. The sort
and the running time (Theorem 2) are not formalized.
-/

namespace Refresh

/-- Day `t` lies in window `p`. -/
def hits (t : Int) (p : Int × Int) : Prop := p.1 ≤ t ∧ t ≤ p.2

/-- The test on line 5: `a > last`, where `none` stands for `last = -∞`. -/
def fresh : Option Int → Int → Bool
  | none, _ => true
  | some l, a => decide (l < a)

/-- The scan of Fig. 1, carrying `last`. Returns the witnesses in order. -/
def refresh : Option Int → List (Int × Int) → List (Int × Int)
  | _, [] => []
  | last, p :: ps => if fresh last p.1 then p :: refresh (some p.2) ps else refresh last ps

/-- The refresh days: one per witness, its closing day. -/
def days (l : List (Int × Int)) : List Int := (refresh none l).map Prod.snd

/-- Every witness is one of the input windows. -/
theorem refresh_sub : ∀ (l : List (Int × Int)) (last : Option Int),
    ∀ w ∈ refresh last l, w ∈ l := by
  intro l
  induction l with
  | nil => intro last w hw; simp [refresh] at hw
  | cons p ps ih =>
    intro last w hw
    simp only [refresh] at hw
    split at hw
    · rcases List.mem_cons.1 hw with rfl | hw
      · exact List.mem_cons_self
      · exact List.mem_cons_of_mem _ (ih _ w hw)
```

```
    · exact List.mem_cons_of_mem _ (ih _ w hw)

/-! ## Lemma 1: every window is covered -/

/-- The invariant behind Lemma 1: if `last` is at most every remaining closing
day, every remaining window holds `last` or the closing day of a witness. -/
theorem covers_aux : ∀ (l : List (Int × Int)) (last : Option Int),
    l.Pairwise (fun p q => p.2 ≤ q.2) → (∀ p ∈ l, p.1 ≤ p.2) →
    (∀ L, last = some L → ∀ p ∈ l, L ≤ p.2) →
    ∀ q ∈ l, (∃ L, last = some L ∧ hits L q) ∨ ∃ w ∈ refresh last l, hits w.2 q := by
  intro l
  induction l with
  | nil => intro last _ _ _ q hq; simp at hq
  | cons p ps ih =>
    intro last hs hwf hL q hq
    rw [List.pairwise_cons] at hs
    have hwf' : ∀ r ∈ ps, r.1 ≤ r.2 := fun r hr => hwf r (List.mem_cons_of_mem _ hr)
    have hp : p.1 ≤ p.2 := hwf p List.mem_cons_self
    simp only [refresh]
    split
    · -- `a > last`: the scan adds `p.2` and continues with `last = p.2`.
      rcases List.mem_cons.1 hq with rfl | hq'
      · exact Or.inr ⟨q, List.mem_cons_self, hp, Int.le_refl _⟩
      · have hL' : ∀ L, some p.2 = some L → ∀ r ∈ ps, L ≤ r.2 := by
          intro L h r hr
          cases h
          exact hs.1 r hr
        rcases ih (some p.2) hs.2 hwf' hL' q hq' with ⟨L, h, hq2⟩ | ⟨w, hw, hq2⟩
        · cases h
          exact Or.inr ⟨p, List.mem_cons_self, hq2⟩
        · exact Or.inr ⟨w, List.mem_cons_of_mem _ hw, hq2⟩
    · -- `a ≤ last`: `last` already lies in `p`.
      next hf =>
      rcases List.mem_cons.1 hq with rfl | hq'
      · cases last with
        | none => simp [fresh] at hf
        | some L =>
          simp [fresh] at hf
          exact Or.inl ⟨L, rfl, hf, hL L rfl q List.mem_cons_self⟩
      · exact ih last hs.2 hwf' (fun L h r hr => hL L h r (List.mem_cons_of_mem _ hr)) q hq'

/-- **Lemma 1.** On input sorted by closing day, every window contains a
refresh day. -/
theorem refresh_covers (l : List (Int × Int))
    (hs : l.Pairwise (fun p q => p.2 ≤ q.2)) (hwf : ∀ p ∈ l, p.1 ≤ p.2) :
    ∀ q ∈ l, ∃ t ∈ days l, hits t q := by
  intro q hq
  rcases covers_aux l none hs hwf (by simp) q hq with ⟨L, h, _⟩ | ⟨w, hw, h⟩
  · cases h
  · exact ⟨w.2, List.mem_map.2 ⟨w, hw, rfl⟩, h⟩

/-! ## Lemma 2: the witnesses are pairwise disjoint -/

/-- Each witness opens after the previous one closes, and after `last`. -/
theorem refresh_separated : ∀ (l : List (Int × Int)) (last : Option Int),
    (∀ p ∈ l, p.1 ≤ p.2) →
    (refresh last l).Pairwise (fun w v => w.2 < v.1) ∧
      ∀ L, last = some L → ∀ w ∈ refresh last l, L < w.1 := by
  intro l
  induction l with
  | nil => intro last _; simp [refresh]
  | cons p ps ih =>
    intro last hwf
    have hwf' : ∀ r ∈ ps, r.1 ≤ r.2 := fun r hr => hwf r (List.mem_cons_of_mem _ hr)
    have hp : p.1 ≤ p.2 := hwf p List.mem_cons_self
    simp only [refresh]
    split
    · next hf =>
      obtain ⟨hpw, hlt⟩ := ih (some p.2) hwf'
      refine ⟨List.pairwise_cons.2 ⟨fun v hv => hlt p.2 rfl v hv, hpw⟩, ?_⟩
      intro L h w hw
      cases h
      simp [fresh] at hf
      rcases List.mem_cons.1 hw with rfl | hw
      · exact hf
      · have := hlt p.2 rfl w hw
        omega
    · exact ih last hwf'

/-- **Lemma 2.** No day lies in two witnesses. -/
theorem witnesses_pairwise_disjoint (l : List (Int × Int)) (hwf : ∀ p ∈ l, p.1 ≤ p.2) :
    (refresh none l).Pairwise (fun w v => ∀ t, ¬ (hits t w ∧ hits t v)) :=
  (refresh_separated l none hwf).1.imp fun h t ⟨⟨_, h1⟩, ⟨h2, _⟩⟩ => by omega
```

```
/-! ## Theorem 1: no set of days that hits every window is smaller -/

/-- Pairwise disjoint windows that are all hit by `T` need `|T|` at least
their number: each needs a day of its own. -/
theorem disjoint_le (W : List (Int × Int)) :
    W.Pairwise (fun w v => ∀ t, ¬ (hits t w ∧ hits t v)) →
    ∀ T : List Int, (∀ w ∈ W, ∃ t ∈ T, hits t w) → W.length ≤ T.length := by
  induction W with
  | nil => intros; simp
  | cons w ws ih =>
    intro hd T hT
    rw [List.pairwise_cons] at hd
    obtain ⟨t, ht, htw⟩ := hT w List.mem_cons_self
    -- The day `t` serves `w` only, so the rest are hit by `T` without `t`.
    have hrest : ∀ v ∈ ws, ∃ s ∈ T.erase t, hits s v := by
      intro v hv
      obtain ⟨s, hs, hsv⟩ := hT v (List.mem_cons_of_mem _ hv)
      have hne : s ≠ t := by
        rintro rfl
        exact hd.1 v hv s ⟨htw, hsv⟩
      exact ⟨s, (List.mem_erase_of_ne hne).2 hs, hsv⟩
    have := ih hd.2 (T.erase t) hrest
    rw [List.length_erase_of_mem ht] at this
    have := List.length_pos_of_mem ht
    simp only [List.length_cons]
    omega

/-- **Theorem 1.** Every list of days that hits every window is at least as
long as the list `refresh` returns. -/
theorem refresh_optimal (l : List (Int × Int))
    (hwf : ∀ p ∈ l, p.1 ≤ p.2) (T : List Int) (hT : ∀ p ∈ l, ∃ t ∈ T, hits t p) :
    (days l).length ≤ T.length := by
  rw [days, List.length_map]
  exact disjoint_le _ (witnesses_pairwise_disjoint l hwf) T
    (fun w hw => hT w (refresh_sub l none w hw))

end Refresh
```

APPENDIX F
LLM USAGE: GREEDY PART

Tools. (1) A GPT-6 Astra brainstorming session that utilized back and forth questioning to get a rough concept of a plan. (2) A GPT-6 Astra thread that reviewed and revised the project prompt. (3) This implementation session: Claude Code (Anthropic's command-line coding agent), model Claude Opus 5.5 (`claude-opus-5-5`), which wrote the code, reviewed the experiments, and revised the draft report and Section II.

Prompt (verbatim). The project implementation session was driven by the single prompt below, reproduced exactly.

Listing 7. Implementation prompt

```
Review the concept md file and rough draft plan. Work in the order given, keep each step inside its time box, and when
     something cannot be finished, mark it clearly rather than silently skipping it.

## Hard rules
1. **Do not edit Ilan's files** except these four, and only as described: `divide_and_conquer/main.tex`, `
     divide_and_conquer/refs.bib` (append greedy citations), `divide_and_conquer/main.pdf` (overwritten by rebuilding,
     never hand-edited), and the top-level `README.md`. Everything else of mine goes under sibling folder `greedy/`.
2. **Self-contained.** `C:\Users\R\Documents\Cruise-Finder` is a read-only reference. Read `cruise_finder/carnival.py` and
      `cruise_finder/config.py` to double check how the Carnival request is built and paginated (URL, query parameters,
     headers, month code, pagination), but do not import from that repo, do not copy its database or application code, do
     not build any app functionality, and do not modify it.
3. Windows machine, PowerShell. Python 3.12 is at `C:\Users\R\AppData\Local\Programs\Python\Python312`. No TeX
     distribution and no `elan`/`lake` are installed; steps 6 and 9 say what to install.


## The problem

**What is fixed:** the abstraction, the algorithm, and the proofs below. Do not change them.

**What is a modeling choice, and must be presented as one:** the application framing. Do not claim anything about how
     cruise prices move unless you cite a source for it and ask me. If a source contradicts a claim below, drop the claim
     and alert me.

**Problem:** Cruise Finder is my self-hosted web app that tracks Carnival Cruise Line sailings from Florida. It gets
     prices from Carnival's public but undocumented search endpoint `GET https://www.carnival.com/cruisesearch/api/search`,
      which the app calls once per configured departure month, following pagination, and only on demand; every call is
     logged in an append-only audit table because the endpoint is unofficial and rate-limited. One **refresh batch**
     queries every configured month and follows every page, so it observes the current price of every listed sailing.
     Assumption to state: we treat every batch as having the same cost (pagination can change over time; the objective
     counts batches, not HTTP requests).
```

The feature we propose: the app accepts **quote requests**. A quote request names a sailing and a date interval [a_i, b_i] during which at least one fresh observation is acceptable. The objective is to satisfy every request with the fewest batches. For the case study, we generate example request windows from sailing dates by an explicitly stated policy: from Carnival's final-payment deadline (76 days before departure for cruises of 5 nights or fewer, 91 days for 6 nights or more; cite Carnival's own page, https://help.carnival.com/app/answers/detail/a_id/481, and verify the numbers there) to booking close, c days before departure.

Question. Given every quote request's window, what is the fewest refresh batches that give every window at least one observation, and on which days?

Abstraction. Days are integers. Window W_i = [a_i, b_i] with a_i = d_i − F(L_i), b_i = d_i − c, F(L) = 76 if L ≤ 5 else 91, c a parameter (default 2; use Carnival's published cutoff if you find one). Drop windows with b_i < today; clip a_i to today. A refresh batch on day t observes every window with t ∈ W_i. Input: n integer intervals with a_i ≤ b_i. Output: a set T of days hitting every interval, |T| minimum. This is the minimum point cover of intervals (interval stabbing). It is **not** interval scheduling; say so in one sentence.

Algorithm REFRESH-DAYS. Sort windows by b ascending. T ← ∅, last ← −∞. For each window in order: if a_i > last, add b_i to T and set last ← b_i. Return T and the windows that triggered an addition (the "witnesses").

Proofs (typeset as Lemma 1, Lemma 2, Theorem 1, Theorem 2 in the style of `sec_dc.tex`):
- Lemma 1 (every window is covered). At window i, either a_i > last and b_i ∈ W_i is added, or a_i ≤ last = b_j for an earlier j with b_j ≤ b_i, so a_i ≤ b_j ≤ b_i and b_j ∈ T lies in W_i.
- Lemma 2 (witnesses are pairwise disjoint). Let i_1 < ... < i_m be the triggering indices. For r < s, when i_s is processed, last = b_{i_{s−1}} ≥ b_{i_r} (last only increases, b's are sorted) and a_{i_s} > last ≥ b_{i_r}, so every day of W_{i_s} exceeds every day of W_{i_r}.
- Theorem 1 (optimality). Feasibility is Lemma 1. Any feasible T needs a day in each of the m pairwise-disjoint witness windows, and one day lies in at most one of them, so |T| ≥ m = |T|.
- Theorem 2 (running time). O(n log n) time with a comparison sort and O(n) extra space; O(n) if the windows arrive sorted by closing day.

Worked example: windows [1,10], [4,12], [8,20], [15,25], [22,30] → days 10 and 25; one day is impossible because [1,10] and [22,30] share no day.

Domain translation (adapt, keep the voice): look at whichever sailing's quote window closes soonest among those not yet observed and refresh on that day; that batch also observes every other sailing whose window is open that day; cross them off and repeat. No schedule with fewer batches exists because the sailings you refreshed *for* have windows that never overlap, so each needed its own day. Finish with the real numbers from step 3, and say in the same breath that each sailing is observed once in its window, not nightly.

Deliverables (exact paths)

```
greedy/
  README.md                 # same shape as divide_and_conquer/README.md
  .gitignore                # __pycache__/, .lake/, LaTeX build files (copy the root .gitignore patterns)
  refresh_days.py           # windows(sailings, today, c) and refresh_days(intervals) -> (days, witnesses)
  test_refresh_days.py
  fetch_sailings.py         # writes data/sailings.csv
  experiments.py            # writes results/*.csv
  plots.py                  # writes figures/*.pdf and *.png
  sec_greedy.tex
  app_greedy.tex            # created as a stub in step 5, filled in step 7
  appendix/transcript.md    # one-line placeholder; I export the transcripts myself
  data/sailings.csv
  results/scaling.csv, results/scan_only.csv
  figures/scaling.pdf/.png, figures/ratio.pdf/.png, figures/scan.pdf/.png, figures/windows.pdf/.png
  proof/                    # step 9 only, mirrors divide_and_conquer/proof
```

Steps, in order

Step 1 Algorithm. `refresh_days.py` with a module docstring like `quadtree.py`'s. `refresh_days(intervals)` takes a list of `(a, b)` integer pairs and returns `(days, witnesses)`. `windows(sailings, today, c)` takes rows with `departure` (date) and `nights` (int) and returns clipped intervals as day offsets from `today`, dropping windows already closed. Raise `ValueError` on a > b.

Step 2 Tests. `test_refresh_days.py`, unittest, run as `python -m unittest test_refresh_days` from `greedy/`:
1. every input interval contains a returned day (Lemma 1);
2. witnesses are pairwise disjoint (Lemma 2);
3. brute force on 300 random instances with n ≤ 10 and endpoints in [0, 20]: enumerate subsets of days 0..20 and assert no feasible subset is smaller than the greedy's. This checks the implementation on small cases; it is not the proof, and the report must not call it one;
4. the worked example returns {10, 25};
5. `windows()` gives 76- and 91-day windows ending c days before departure for a 4-night and an 8-night sailing, and drops a sailing whose window closed yesterday.

All tests must pass before you continue.

Step 3 Real data. `fetch_sailings.py`: for each month November 2026 through April 2027, request page 1 with ports `MIA,PCV,TPA,JAX` and page size 200, using the same query shape, headers, and `MMYYYY` month code as Cruise-Finder's `carnival.py`, then **follow every page the response reports** (`lastPage`), exactly as the reference does. Sleep 3 seconds between requests. Write `data/sailings.csv` with columns `sailing_id, ship, port, itinerary, departure, nights` (nights from `dur`; confirm against arrival − departure). Then build windows with today = 2026-10-26 and c = 2, run

```
 the greedy, and print n, m, the chosen dates, and the horizon length in days. Those numbers go in the domain
paragraph and the README. **If the endpoint errors, returns a block page, or returns zero itineraries for every month,
 stop and ask me.** Do not fabricate data.

**Step 4 Experiments.** `experiments.py`: n = 2^10 ... 2^22. Generate windows by drawing departure days uniformly over a
horizon of **2n days** (so closing days are mostly distinct; a fixed 730-day horizon would give millions of ties and
measure something else) and lengths from the empirical length mix in `data/sailings.csv`, c = 2. Five runs per n on
fresh inputs, fastest kept, gc disabled, Ilan's protocol. Also time the scan alone on pre-sorted input. Assert
feasibility (Lemma 1) and witness count = output size on every run. Write `results/scaling.csv` and `results/scan_only
.csv`. `plots.py`: (a) time vs n with an n log n least-squares fit; (b) time/(n log n); (c) scan-only time vs n with a
 linear fit; (d) `windows.pdf`: the real sailings as horizontal bars sorted by closing day, chosen refresh days as
vertical lines, witnesses highlighted. In the report, describe what the plots show; Theorem 2 is an upper bound, so
say whether the ratio is flat rather than asserting it must be. Record the CPU (`Get-CimInstance Win32_Processor |
Select Name`) and Python version.

**Step 5 The section and main.tex.** `sec_greedy.tex`, structured like `sec_dc.tex` (problem, abstraction, algorithm with
an `algorithmic` figure, running time, correctness with lemma/theorem/IEEEproof environments, explanation in the
problem domain, experiments with Setup and Running-time paragraphs and the figures). Section title `Greedy: Fewest
Price Refreshes Under a Request Budget`, label `sec:greedy`. State the modeling choices and the limitation from "The
problem" above. Citations, appended to `divide_and_conquer/refs.bib`: Carnival's final-payment page (above); Carnival'
s booking cutoff only if you find Carnival's own page, otherwise c stays an assumption; an industry source on cabins
returning to inventory / repricing after final payment only if found; the Cruise-Finder repository (read the remote
from that folder's `.git/config`; if private, cite as "private repository, available on request"); Kleinberg & Tardos
chapter 4 for the greedy paradigm. No rambling.

Create `app_greedy.tex` now as a stub containing the two appendix section headings and labels (`app:greedy-code`, `app:
greedy-llm`) so main.tex compiles in step 6; fill it in step 7.

Edit `divide_and_conquer/main.tex`: replace **every** `\partnertodo{...}`: author block (Ryan Rodriguez, rrodriguez7@ufl.
edu, same affiliation lines as Ilan), abstract sentences, keywords (`greedy algorithm, interval point cover, web
scraping, rate limiting, cruise fares`), the introduction paragraph (both halves scrape travel marketplaces under a
platform limit: the divide-and-conquer half asks *where* to look when a map caps results per window; this half asks *
when* to look when an API caps how often you may ask), the greedy section (replace the placeholder with `\input{../
greedy/sec_greedy}`, keep `\input{sec_dc}` after it), and the conclusion paragraph. Reference greedy figures and
listings with `../greedy/` paths. Add `\input{../greedy/app_greedy}` after the existing divide-and-conquer appendix
inputs. Do not restructure anything else.

**Step 6 Compile.** Install Tectonic: `winget install --id Tectonic.Tectonic`; if that id fails, `winget search tectonic`;
 if winget is unavailable, install MiKTeX basic via winget and use `pdflatex` + `bibtex` + `pdflatex` ×2. Compile
from `divide_and_conquer/` with `tectonic main.tex`. Fix every error and every undefined reference or citation warning
. Verify: `rg -n "partnertodo" divide_and_conquer/main.tex divide_and_conquer/sec_dc.tex greedy/*.tex` returns only
the `\newcommand` line; `rg -n "\?\?" divide_and_conquer/main.log` is empty (if Tectonic keeps no log, grep its
console output). Overwrite the tracked `divide_and_conquer/main.pdf` and report its page count. Confirm no absolute
paths appear in any `.tex` so the folder still builds on Overleaf.

**Step 7 Appendix and README.** Fill `app_greedy.tex`: Appendix "Greedy Code" with `\lstinputlisting` of `refresh_days.py`
 and `experiments.py` (tests, plotting script, data, and raw results referenced as in the repository, like `app_dc.tex
`), then Appendix "Greedy LLM Prompts and Results" in the format of `app_dc.tex`: tools and models (Claude Code with
the model you are running as, for implementation; an earlier Claude Code planning session and a Codex session that
researched the course requirements and reviewed the plan, whose transcripts I will add), this entire prompt verbatim,
then per step a short record of what you produced, what you checked, and every correction (failed test, compile error,
 proof attempt that did not close).

**Step 8. Verification and report.** From `greedy/`: `python -m unittest test_refresh_days`; confirm the CSVs and figures
exist; `python plots.py`. From the repo root: `git status --short`; it must list only `greedy/**`, `divide_and_conquer
/main.tex`, `divide_and_conquer/main.pdf`, `divide_and_conquer/refs.bib`, and `README.md`. Then report: real-data
numbers (n, m, dates); page count; citations found vs left as assumptions; anything unfinished or unverified.

**Step 9  Lean formalization.** Install elan with the official Windows installer from https://github.com/leanprover/elan,
confirm `cd divide_and_conquer/proof; lake build` succeeds so toolchain `leanprover/lean4:v4.35.0` is present, then
create `greedy/proof/` mirroring Ilan's layout (`lakefile.toml`, same `lean-toolchain`, `Proof.lean`, `Proof/Refresh.
lean`, `.gitignore` for `.lake/`). Core Lean only, no Mathlib. Model intervals as `Int × Int`, the input as `List (
Int × Int)` with hypotheses `List.Pairwise (fun p q => p.2 ≤ q.2)` and `∀ p ∈ l, p.1 ≤ p.2`. Define `refresh` by
structural recursion carrying `last`. Prove in order: (1) `refresh_covers`; (2) `witnesses_pairwise_disjoint`; (3) `
refresh_optimal` (any covering list of days is at least as long). Add a Lean table and listing to `sec_greedy.tex` / `
app_greedy.tex` the way Ilan's half does, saying exactly which statements are machine-checked. **Recompile `main.pdf`,
 rerun the step 8 checks, and update the report.**

## Stop and ask (only these)

- Carnival's endpoint blocks or returns nothing for every month in step 3.
- Tectonic and MiKTeX both fail to install.
- Anything that would require editing one of Ilan's files beyond the three allowed.
```

Results, by step. (1) `refresh_days.py`: `refresh_days` returns the days and the witnesses; `windows` builds clipped windows from sailing dates; a separate `refresh_days_sorted` holds the scan alone for the pre-sorted experiment. (2) `test_refresh_days.py` with the five requested checks and a check that a reversed interval raises `ValueError`. All six tests passed on the first run. (3) `fetch_sailings.py` downloaded six months of sailings on October 26, 2026: one page per month (no month exceeded 200 itineraries), 383 sailings, every duration equal to arrival minus departure. With $c = 2$: $n = 383$ windows, $m = 3$ refresh days (October 30, 2026; January 14, 2027; March 30, 2027), horizon 184 days.

Carnival's help page returned HTTP 403 to scripted requests and never finished loading in a browser, so the 76- and 91-day deadlines were read from the Internet Archive's copy of the same page (July 23, 2025), cited in [4]. That table counts cruise length in days; the session checked that the "N-Day" itinerary titles, the duration field, and arrival minus departure agree on all 383 sailings, so the 5-or-fewer rule applies to the duration field directly. (4) `experiments.py` and `plots.py`, about one minute to run. The full algorithm fit $c\,n \log_2 n$ with $R^2 = 0.998$. The scan alone grew from 20 to 117 ns per window above $n = 2^{17}$; a follow-up timing that re-allocated the sorted tuples in memory order reduced this to 70 ns at $n = 2^{22}$, which supports, but does not prove, the cache explanation in Section II-G. After the first compile, the least-squares line in Fig. 2(c) sat above every in-cache point because the large sizes dominated it, so the fit was restricted to $n \le 2^{17}$ and the caption says so; legends that covered data points were moved. (5) Section II, the bibliography entries, and the front matter, introduction and conclusion in `main.tex`. (6) Tectonic is not in the `winget` catalog (both the package id and a search returned nothing), so MiKTeX was installed with `winget`, with on-the-fly package installation enabled. (7) This appendix, `greedy/README.md` and the top-level `README.md`. (8) Verification: unit tests, presence of every output file, and `git status` listing only the permitted paths. (9) Lean formalization. The official `elan` installer was run (its first invocation failed on a PowerShell argument-type error and was rerun); `lake build` of the divide-and-conquer proof succeeded, installing Lean 4.35.0. `greedy/proof/Proof/Refresh.lean` defines the scan with *last* as an `Option Int` argument (`none` for $-\infty$) returning the witnesses, and proves `refresh_covers`, `witnesses_pairwise_disjoint` and `refresh_optimal`.

Full transcripts. The prompts, tool calls and responses of every session, in order, are in the repository at `greedy/appendix/transcript.md`. To the best of my knowledge they comply with the LLM policies of the course.